

TÀI LIỆU HỌC SOURCE CODE

THIẾT KẾ VÀ MÔ PHỎNG HỆ THỐNG GIÁM SÁT MÔI TRƯỜNG VÀ ĐIỀU KHIỂN THIẾT BỊ NHÀ THÔNG MINH THỜI GIAN THỰC SỬ DỤNG ESP32 VÀ FREERTOS

TÀI LIỆU HỌC SOURCE CODE - ESP32-S3 + FreeRTOS + FSM / Rule-Based Control

PROJECT FINAL

SensorTask -> sensorQueue -> DecisionTask -> decisionQueue -> ControlTask

PIR -> Hardware Interrupt -> ISR -> Semaphore -> PIRTask

DisplayTask -> OLED

WebTask -> HTTP Web Dashboard

Mục tiêu: đọc source, hiểu cách chạy, và chuẩn bị bảo vệ đồ án.

MỤC LỤC HỌC NHANH

- 1. Kiến trúc tổng thể và cách đọc project
- 2. main.cpp - khởi động ESP32-S3 và tạo FreeRTOS Task
- 3. core/config.h và core/types.h - cấu hình và kiểu dữ liệu
- 4. core/app_state - Queue, Semaphore, Mutex và trạng thái dùng chung
- 5. sensors - đọc DHT22, LDR và nguồn WEB
- 6. decision - FSM / Rule-Based Control
- 7. control - Relay, Fan PWM và Buzzer
- 8. pir - Hardware Interrupt, ISR, Semaphore và PIRTask
- 9. display - OLED và nguyên tắc tách giao diện khỏi control
- 10. web - HTTP server, REAL/WEB mode và Web Dashboard
- 11. FreeRTOS Scheduler, priority và luồng chạy thực tế
- 12. Đánh giá real-time: latency, response time, execution time
- 13. Các kịch bản chạy mẫu
- 14. Điểm cần chú ý và hướng cải tiến source
- 15. Bộ câu hỏi tự kiểm tra và câu trả lời bảo vệ gần

Giải thích bản chất: Đừng cố nhớ từng dòng code. Hãy nhớ theo chuỗi SENSE -> DECIDE -> ACT. Sau đó học thêm đường sự kiện PIR và đường giao diện Web/OLED.

1. KIẾN TRÚC TỔNG THỂ VÀ CÁCH ĐỌC PROJECT

Project được chia module theo trách nhiệm. Mỗi folder trả lời một câu hỏi riêng: cảm biến lấy dữ liệu ở đâu, FSM quyết định thế nào, actuator được điều khiển ở đâu, interrupt PIR được xử lý ra sao và Web giao tiếp với ESP32 bằng cách nào.

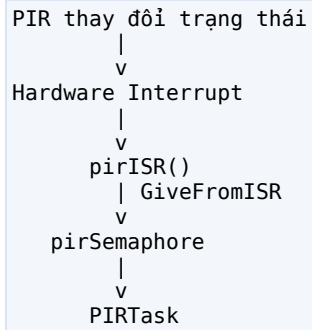
```
src/  
|-- main.cpp  
|-- core/  
|   |-- config.h  
|   |-- types.h  
|   |-- app_state.h  
|   |-- app_state.cpp  
|-- sensors/  
|-- decision/  
|-- control/  
|-- pir/  
|-- display/  
|-- web/
```

Module	Vai trò chính
main.cpp	Khởi tạo hardware, tạo RTOS object và tạo task
core/	GPIO, threshold, kiểu dữ liệu, Queue/Semaphore/Mutex và metrics
sensors/	Đọc DHT22, LDR hoặc dữ liệu từ Web
decision/	FSM / Rule-Based: SensorData -> SystemMode
control/	SystemMode -> Relay / Fan PWM / Buzzer
pir/	Interrupt -> ISR -> Semaphore -> PIRTask
display/	Hiển thị OLED
web/	Wi-Fi, HTTP API, Web Dashboard, REAL/WEB mode

1.1. Luồng dữ liệu chính

```
DHT22 / LDR / PIR / WEB  
|  
v  
SensorTask  
| SensorData  
v  
sensorQueue  
|  
v  
DecisionTask  
| evaluateMode()  
v  
SystemMode  
|  
v  
decisionQueue  
|  
v  
ControlTask  
/ | \  
Relay Fan Buzzer
```

1.2. Đường sự kiện PIR



Giải thích bản chất: Runtime của hệ thống không chạy theo thứ tự file. Source được chia file để dễ bảo trì, còn FreeRTOS Scheduler quyết định task nào được CPU chạy tại từng thời điểm.

2. main.cpp - KHỞI ĐỘNG ESP32-S3 VÀ TẠO FREERTOS TASK

main.cpp được cố ý giữ ngắn. Nó không chứa logic cảm biến, FSM hay Web chi tiết. Nhiệm vụ của main.cpp chỉ là ghép các module lại thành một hệ thống.

2.1. Include các module

```
#include "core/config.h"
#include "core/app_state.h"
#include "sensors/sensors.h"
#include "decision/decision.h"
#include "control/control.h"
#include "pir/pir.h"
#include "display/display.h"
#include "web/web_server.h"
```

Ý nghĩa: main.cpp chỉ cần biết giao diện hàm của từng module. Chi tiết thực thi nằm trong các file .cpp tương ứng.

2.2. setup() chạy một lần khi ESP32 khởi động

```
initSensors();
initControl();
initDisplay();

currentOccupancy = digitalRead(PIR_PIN);

if (!initRTOSObjects()) {
    ...
}

initPIRInterrupt();
```

1. Khởi tạo sensor và GPIO input.
2. Khởi tạo relay, PWM fan, buzzer và OLED.
3. Tạo Queue, Semaphore và Mutex.
4. Sau khi pirSemaphore đã tồn tại mới gắn hardware interrupt PIR.

2.3. Tạo task và priority

Task	Priority	Chức năng
PIRTask	4	Xử lý sự kiện PIR
ControlTask	3	Cập nhật actuator
SensorTask	2	Đọc cảm biến / Web input
DecisionTask	2	Chạy FSM / Rule
DisplayTask	1	Cập nhật OLED
WebTask	1	Phục vụ HTTP Dashboard

```
xTaskCreate(PIRTask, "PIRTask", 2048, nullptr, 4, nullptr);
xTaskCreate(ControlTask, "ControlTask", 4096, nullptr, 3, nullptr);
xTaskCreate(SensorTask, "SensorTask", 4096, nullptr, 2, nullptr);
xTaskCreate(DecisionTask, "DecisionTask", 3072, nullptr, 2, nullptr);
xTaskCreate(DisplayTask, "DisplayTask", 3072, nullptr, 1, nullptr);
xTaskCreate(WebTask, "WebTask", 6144, nullptr, 1, nullptr);
```

2.4. Tại sao loop() gần như rỗng?

```
void loop() {  
    vTaskDelay(pdMS_TO_TICKS(1000));  
}
```

Giải thích bản chất: Logic chính không nằm trong Arduino loop(). Sau setup(), từng FreeRTOS task tự chạy vòng lặp riêng. Scheduler quản lý CPU giữa các task.

3. core/config.h VÀ core/types.h

3.1. GPIO mapping

Thiết bị	GPIO	Loại tín hiệu
DHT22	15	Digital sensor data
LDR	4	ADC input
PIR	16	Digital input + interrupt
Relay / Light	6	Digital output
Fan PWM	17	PWM output
Buzzer	7	Tone / digital output
OLED SDA	8	I2C SDA
OLED SCL	9	I2C SCL

3.2. Threshold của FSM

```
constexpr float TEMP_WARNING = 45.0f;  
constexpr float TEMP_COOLING = 30.0f;  
constexpr float TEMP_FAN_MAX = 35.0f;  
constexpr float LIGHT_DARK = 30.0f;
```

Các threshold này là ngưỡng thiết kế. Thay đổi ngưỡng chỉ cần sửa config.h, không phải tìm trong nhiều module.

3.3. Chu kỳ task

Thông số	Giá trị
Sensor REAL	2000 ms
Sensor WEB	500 ms
DecisionTask	100 ms
ControlTask	200 ms
DisplayTask	500 ms
Control threshold để theo dõi execution	200 ms

3.4. SensorData - gói dữ liệu cảm biến

```
struct SensorData {  
    float temperature;  
    float humidity;  
    float light;  
    bool occupancy;  
    TickType_t sampleTick;  
};
```

Giải thích bản chất: Thay vì nhiều task cùng đọc bốn biến global riêng lẻ, SensorData đóng gói chúng thành một cấu trúc hoàn chỉnh. sampleTick ghi thời điểm tạo mẫu để đo response time.

3.5. SystemMode và InputSource

```
enum SystemMode {  
    COMFORT,  
    COOLING,  
    LIGHTING,  
    ENERGY_SAVING,  
    WARNING  
};  
  
enum InputSource {  
    REAL_SENSOR,  
    WEB_SENSOR  
};
```

Lưu ý: Humidity được thu thập và hiển thị nhưng trong phiên bản source hiện tại chưa được dùng trong evaluateMode(). Nếu bị hỏi khi bảo vệ, phải trả lời đúng điều này.

4. core/app_state - QUEUE, SEMAPHORE, MUTEX VÀ SHARED STATE

Module app_state chứa các đối tượng FreeRTOS và những trạng thái mà nhiều module cùng cần truy cập.

```
QueueHandle_t sensorQueue;  
QueueHandle_t decisionQueue;  
SemaphoreHandle_t pirSemaphore;  
SemaphoreHandle_t webMutex;
```

4.1. sensorQueue

```
sensorQueue = xQueueCreate(1, sizeof(SensorData));
```

Queue có chiều dài 1. SensorTask dùng xQueueOverwrite(), nên sensorQueue hoạt động như một hộp thư luôn giữ mẫu mới nhất.

```
Mẫu cũ: [T=25, H=55, L=70, O=1]  
      |  
      xQueueOverwrite(new)  
      v  
Mẫu mới: [T=34, H=60, L=20, O=1]
```

4.2. decisionQueue

```
decisionQueue = xQueueCreate(1, sizeof(DecisionData));
```

Nó lưu SystemMode mới nhất để ControlTask và DisplayTask đọc.

4.3. pirSemaphore

```
pirSemaphore = xSemaphoreCreateBinary();
```

Giải thích bản chất: Binary semaphore ở đây giống một chuông báo sự kiện: ISR không truyền dữ liệu phức tạp, nó chỉ báo cho PIRTask rằng PIR vừa thay đổi.

4.4. webMutex

```
webMutex = xSemaphoreCreateMutex();
```

webData có thể được WebTask ghi trong lúc SensorTask đọc. Mutex bảo vệ đoạn critical section để tránh đọc một cấu trúc đang cập nhật dở.

```
WebTask:  LOCK -> ghi webData -> UNLOCK  
SensorTask: LOCK -> copy webData -> UNLOCK
```

4.5. volatile dùng ở đâu?

Một số biến như currentOccupancy, inputSource và timing metrics được khai báo volatile vì giá trị có thể thay đổi ngoài luồng đọc thông thường của một đoạn code. volatile yêu cầu compiler không giả định giá trị luôn giữ nguyên trong thanh ghi.

Lưu ý: volatile không thay thế Mutex hay Queue. Nó không tự làm thao tác đọc/ghi trở thành thread-safe.

5. sensors - ĐỌC DHT22, LDR VÀ NGUỒN WEB

5.1. Khởi tạo sensor

```
void initSensors() {  
    dht.setup(DHT_PIN, DHTesp::DHT22);  
    pinMode(LDR_PIN, INPUT);  
    pinMode(PIR_PIN, INPUT);  
}
```

5.2. SensorTask có hai nguồn dữ liệu

```
if (inputSource == WEB_SENSOR) {  
    ...  
    continue;  
}  
  
// nếu không phải WEB -> REAL SENSOR
```

5.3. WEB mode

```
if (xSemaphoreTake(webMutex, pdMS_TO_TICKS(50)) == pdTRUE) {  
    data = webData;  
    xSemaphoreGive(webMutex);  
}  
  
data.sampleTick = xTaskGetTickCount();  
xQueueOverwrite(sensorQueue, &data);  
vTaskDelay(pdMS_TO_TICKS(SENSOR_WEB_PERIOD_MS));
```

Giải thích bản chất: Web mode không bypass FSM. Browser chỉ tạo SensorData giả lập; dữ liệu vẫn đi qua sensorQueue -> DecisionTask -> ControlTask giống dữ liệu sensor thật.

5.4. REAL mode - DHT22

```
TempAndHumidity th = dht.getTempAndHumidity();  
  
if (!isnan(th.temperature)) data.temperature = th.temperature;  
if (!isnan(th.humidity))    data.humidity = th.humidity;
```

Nếu DHT22 trả NaN, source không ghi đè giá trị đang có. Đây là cơ chế tối thiểu để tránh đưa dữ liệu lỗi vào pipeline.

5.5. REAL mode - LDR qua ADC

```
int raw = analogRead(LDR_PIN);  
data.light = (raw / 4095.0f) * 100.0f;
```

ADC 12 bit có miền xấp xỉ 0..4095. Source ánh xạ sang phần trăm 0..100%. Đây là mức sáng tương đối trong mô phỏng, không phải lux tuyệt đối.

5.6. Occupancy

```
data.occupancy = currentOccupancy;
```

currentOccupancy được PIRTask cập nhật khi có interrupt. Nhờ vậy SensorTask không phải liên tục polling PIR.

5.7. Đóng gói và gửi Queue

```
data.sampleTick = xTaskGetTickCount();  
xQueueOverwrite(sensorQueue, &data);  
vTaskDelay(pdMS_TO_TICKS(SENSOR_REAL_PERIOD_MS));
```

6. decision - FSM / RULE-BASED CONTROL

Đây là bộ não quyết định trong phiên bản điều khiển theo FSM/Rule-Based. Hàm evaluateMode() nhận SensorData và trả về một SystemMode.

```
SystemMode evaluateMode(const SensorData &data) {
    if (data.temperature >= TEMP_WARNING)
        return WARNING;

    if (!data.occupancy)
        return ENERGY_SAVING;

    if (data.temperature >= TEMP_COOLING)
        return COOLING;

    if (data.light < LIGHT_DARK)
        return LIGHTING;

    return COMFORT;
}
```

6.1. Thứ tự ưu tiên

```
Temperature >= 45 ? ----YES----> WARNING
      | NO
      v
Occupancy == NO ? -----> ENERGY_SAVING
      | YES
      v
Temperature >= 30 ? -----> COOLING
      | NO
      v
Light < 30 ? -----> LIGHTING
      | NO
      v
      COMFORT
```

Input ví dụ	Mode
T=50, O=NO, Light=10	WARNING (vì WARNING kiểm tra trước)
T=34, O=NO, Light=10	ENERGY_SAVING
T=34, O=YES, Light=70	COOLING
T=25, O=YES, Light=15	LIGHTING
T=25, O=YES, Light=70	COMFORT

6.2. DecisionTask

```
if (xQueuePeek(sensorQueue, &sensor, pdMS_TO_TICKS(100)) == pdTRUE) {
    decision.mode = evaluateMode(sensor);
    xQueueOverwrite(decisionQueue, &decision);
}

vTaskDelay(pdMS_TO_TICKS(DECISION_PERIOD_MS));
```

Giải thích bản chất: xQueuePeek() đọc nhưng không lấy item ra khỏi Queue. Điều này phù hợp vì nhiều task có thể cần xem mẫu mới nhất.

Lưu ý: FSM priority và FreeRTOS task priority là hai khái niệm khác nhau. FSM priority chọn trạng thái; task priority chọn task nào được CPU ưu tiên khi nhiều task READY.

7. control - RELAY, FAN PWM VÀ BUZZER

7.1. Khởi tạo actuator

```
pinMode(LIGHT_PIN, OUTPUT);
pinMode(BUZZER_PIN, OUTPUT);

ledcSetup(PWM_CHANNEL, PWM_FREQ, PWM_RESOLUTION);
ledcAttachPin(FAN_PIN, PWM_CHANNEL);

digitalWrite(LIGHT_PIN, LOW);
ledcWrite(PWM_CHANNEL, 0);
noTone(BUZZER_PIN);
```

PWM resolution = 8 bit nên duty command nằm trong 0..255. PWM frequency được đặt 5 kHz.

7.2. ControlTask dùng mode để tạo trạng thái output

```
int nextFan = 0;
bool nextLight = false;
bool nextAlarm = false;

switch (decision.mode) {
    ...
}
```

Mode	Light	Fan	Buzzer
COMFORT	OFF	0%	OFF
ENERGY_SAVING	OFF	0%	OFF
LIGHTING	ON	0%	OFF
COOLING	OFF*	~70% hoặc 100%	OFF
WARNING	OFF	100%	ON

* Trong COOLING, nếu có người và Light < 30%, ControlTask vẫn bật đèn để xử lý đồng thời nóng + tối.

7.3. Điều khiển fan khi COOLING

```
nextFan = (sensor.temperature >= TEMP_FAN_MAX) ? 255 : 180;
```

Tương đương: nếu T >= 35°C thì PWM=255 (100%), ngược lại PWM=180 (xấp xỉ 70.6%).

7.4. Ghi xuống phần cứng

```
digitalWrite(LIGHT_PIN, nextLight ? HIGH : LOW);
ledcWrite(PWM_CHANNEL, nextFan);

if (nextAlarm)
    tone(BUZZER_PIN, 1000);
else
    noTone(BUZZER_PIN);
```

Giải thích bản chất: DecisionTask chỉ quyết định MODE. ControlTask mới là nơi thực sự ghi GPIO/PWM. Cách tách này giúp logic quyết định độc lập với driver actuator.

8. pir - HARDWARE INTERRUPT, ISR, SEMAPHORE VÀ PIRTask

8.1. Gắn hardware interrupt

```
attachInterrupt(  
    digitalPinToInterrupt(PIR_PIN),  
    pirISR,  
    CHANGE  
);
```

CHANGE tạo interrupt khi chân PIR đổi LOW->HIGH hoặc HIGH->LOW.

8.2. ISR phải ngắn

```
void IRAM_ATTR pirISR() {  
    BaseType_t taskWoken = pdFALSE;  
  
    pirIsrTick = xTaskGetTickCountFromISR();  
    xSemaphoreGiveFromISR(pirSemaphore, &taskWoken);  
  
    if (taskWoken == pdTRUE)  
        portYIELD_FROM_ISR();  
}
```

Giải thích bản chất: ISR không cập nhật OLED, không chạy HTTP, không đọc DHT22 và không chạy FSM. Nó chỉ ghi timestamp + phát semaphore rồi thoát.

8.3. PIRTask ngủ cho tới khi có event

```
xSemaphoreTake(  
    pirSemaphore,  
    portMAX_DELAY  
);
```

Khi chưa có sự kiện, PIRTask ở trạng thái BLOCKED và không cần polling liên tục.

8.4. Khi semaphore được phát

```
currentOccupancy = digitalRead(PIR_PIN);  
  
TickType_t now = xTaskGetTickCount();  
pirLatencyMs = (now - pirIsrTick) * portTICK_PERIOD_MS;
```

8.5. Cập nhật sensorQueue ngay

```
if (inputSource == REAL_SENSOR) {  
    SensorData sensor;  
  
    if (xQueuePeek(sensorQueue, &sensor, 0) == pdTRUE) {  
        sensor.occupancy = currentOccupancy;  
        sensor.sampleTick = now;  
        xQueueOverwrite(sensorQueue, &sensor);  
    }  
}
```

Điểm này giúp occupancy phản ánh nhanh hơn thay vì phải đợi chu kỳ SensorTask REAL 2 giây.

Lưu ý: Nếu log ISR->PIRTask hiển thị 0 ms, không nên kết luận latency tuyệt đối bằng 0. Phép đo dựa trên FreeRTOS tick nên giá trị có thể nhỏ hơn độ phân giải hiện tại.

9. display - OLED VÀ TÁCH GIAO DIỆN KHỎI CONTROL

```
Wire.begin(OLED_SDA, OLED_SCL);
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

DisplayTask chỉ xQueuePeek() sensorQueue và decisionQueue rồi hiển thị source, T/H, Light, PIR, Mode, Fan và Light state.

```
if (xQueuePeek(sensorQueue, &sensor, 0) == pdTRUE &&
    xQueuePeek(decisionQueue, &decision, 0) == pdTRUE) {
    ... display ...
}

vTaskDelay(pdMS_TO_TICKS(DISPLAY_PERIOD_MS));
```

Giải thích bản chất: OLED có priority 1. Nếu giao diện chậm nhẹ, ControlTask priority 3 vẫn được ưu tiên. Display không được quyền quyết định mode hay bật actuator.

10. web - HTTP SERVER, REAL/WEB MODE VÀ DASHBOARD

10.1. ESP32 chạy WebServer port 80

```
static WebServer server(80);
```

10.2. Kết nối Wokwi Wi-Fi

```
WiFi.begin("Wokwi-GUEST", "", 6);

while (WiFi.status() != WL_CONNECTED) {
    vTaskDelay(pdMS_TO_TICKS(250));
}
```

10.3. HTTP endpoints

Endpoint	Method	Ý nghĩa
/	GET	Trả HTML Dashboard
/api/state	GET	Browser đọc trạng thái hệ thống
/api/sensors	POST	Gửi sensor giả lập và chuyển WEB mode
/api/real	POST	Quay về DHT22/LDR/PIR

10.4. WEB mode không điều khiển fan trực tiếp

```
Browser sliders
| POST /api/sensors
v
SensorData
|
v
sensorQueue
|
v
DecisionTask -> FSM -> ControlTask -> Actuator
```

10.5. Dashboard refresh

```
setInterval(refresh, 500);
```

Browser gọi GET /api/state khoảng mỗi 500 ms và cập nhật System Mode, sensor, actuator và timing metrics.

11. FREERTOS SCHEDULER, PRIORITY VÀ LUỒNG CHẠY THỰC TẾ

Các task không chạy theo kiểu SensorTask chạy hết rồi mới tới DecisionTask rồi mới tới ControlTask một cách cứng nhắc. Scheduler chọn task READY có priority phù hợp.

Task	Priority	Trạng thái thường gặp
PIRTask	4	BLOCKED chờ semaphore; READY ngay khi PIR event
ControlTask	3	READY theo chu kỳ ~200 ms
SensorTask	2	Delay 2 s REAL hoặc 500 ms WEB
DecisionTask	2	Chạy khoảng mỗi 100 ms
DisplayTask	1	Chạy khoảng mỗi 500 ms
WebTask	1	handleClient rồi delay ~2 ms

11.1. Ví dụ tại một thời điểm

```
DisplayTask READY (priority 1)
ControlTask READY (priority 3)
PIR interrupt xảy ra -> PIRTask READY (priority 4)

Scheduler ưu tiên PIRTask trước, sau đó ControlTask, rồi mới tới DisplayTask
(nếu các điều kiện READY vẫn còn).
```

11.2. vTaskDelay không giống bận chờ

Khi task gọi vTaskDelay(), task chuyển sang BLOCKED một thời gian. CPU có thể chạy task khác. Đây là lý do cấu trúc FreeRTOS để tổ chức hơn super loop chứa nhiều delay tuần tự.

11.3. Queue / Semaphore / Mutex khác nhau

Cơ chế	Dùng để	Trong project
Queue	Truyền gói dữ liệu	SensorData, DecisionData
Semaphore	Báo event / đồng bộ	ISR PIR -> PIRTask
Mutex	Bảo vệ shared data	webData

12. ĐÁNH GIÁ REAL-TIME TRONG SOURCE

12.1. PIR ISR -> PIRTask latency

```
pirIsrTick = xTaskGetTickCountFromISR();  
...  
TickType_t now = xTaskGetTickCount();  
pirLatencyMs = (now - pirIsrTick) * portTICK_PERIOD_MS;
```

Đo thời gian từ khi ISR ghi timestamp tới khi PIRTask thực sự được scheduler chạy và lấy timestamp mới.

12.2. Sensor sample -> actuator response time

```
// SensorTask / PIRTask  
sensor.sampleTick = xTaskGetTickCount();  
  
// ControlTask sau khi update actuator  
responseTimeMs = (now - sensor.sampleTick) * portTICK_PERIOD_MS;
```

Đây là metric gần với end-to-end response của pipeline input -> decision -> output.

12.3. ControlTask execution time

```
TickType_t startTick = xTaskGetTickCount();  
... control work ...  
TickType_t endTick = xTaskGetTickCount();  
controlExecMs = (endTick - startTick) * portTICK_PERIOD_MS;
```

12.4. controlDeadlineMiss

```
controlCycles++;  
if (controlExecMs > CONTROL_DEADLINE_MS)  
    controlDeadlineMiss++;
```

Dashboard tính tỷ lệ số chu kỳ không vượt 200 ms.

Lưu ý: Tên "deadline success" trên Dashboard mạnh hơn bản chất phép đo hiện tại. Source đang chủ yếu kiểm tra execution time có vượt 200 ms không, chưa đo đầy đủ absolute real-time deadline gồm release, scheduling delay và completion time. Khi báo cáo nên mô tả là theo dõi ControlTask execution-time threshold.

12.5. Hard real-time hay soft real-time?

Với cấu trúc hiện tại và mô phỏng Wokwi, nên mô tả hệ thống theo hướng real-time task architecture / near-real-time hoặc soft real-time monitoring. Chưa có đủ bằng chứng để khẳng định hard real-time guarantee.

13. CÁC KỊCH BẢN CHẠY MẪU

Case	Input	FSM	Output mong đợi
1	T=25, H=55, Light=70, O=YES	COMFORT	Light OFF, Fan 0%, Alarm OFF
2	T=34, H=60, Light=60, O=YES	COOLING	Light OFF, Fan ~70%, Alarm OFF
3	T=34, H=60, Light=15, O=YES	COOLING	Light ON, Fan ~70%, Alarm OFF
4	T=25, H=55, Light=15, O=YES	LIGHTING	Light ON, Fan 0%, Alarm OFF
5	T=28, H=55, Light=50, O=NO	ENERGY_SAVING	All OFF
6	T=50, H=60, Light=10, O=NO	WARNING	Light OFF, Fan 100%, Alarm ON

13.1. Case COOLING diễn ra thế nào?

Input: T=34, H=60, Light=60, Occupancy=YES

SensorTask -> SensorData -> sensorQueue

DecisionTask:

34 < 45 -> not WARNING

occupancy YES -> not ENERGY_SAVING

34 >= 30 -> COOLING

ControlTask:

34 < 35 -> PWM = 180 (~70.6%)

Light 60 >= 30 -> Light OFF

Alarm OFF

13.2. Case WARNING cho thấy priority FSM

Input: T=50, Light=10, Occupancy=NO

Rule đầu tiên: T >= 45 -> WARNING

Không xét tiếp "không có người".

Output: Fan 100%, Buzzer ON, Light OFF

14. ĐIỂM CẦN CHÚ Ý VÀ HƯỚNG CẢI TIẾN SOURCE

14.1. Humidity chưa tham gia FSM

Hiện humidity được đọc và hiển thị nhưng không tác động mode. Có thể mở rộng rule như cảnh báo độ ẩm quá cao/thấp hoặc kết hợp temperature + humidity cho cooling.

14.2. Sensor và Decision đang ở hai Queue riêng

ControlTask xQueuePeek(sensorQueue) và xQueuePeek(decisionQueue) riêng biệt. Về lý thuyết có một cửa sổ ngắn sensorQueue đã là mẫu mới nhưng decisionQueue vẫn là mode của mẫu trước.

```
Thời điểm A: sensorQueue = sample #10  
             decisionQueue = decision của sample #9
```

```
Một thiết kế chặt hơn có thể dùng:  
SystemState { SensorData sensor; SystemMode mode; ... }
```

14.3. WebTask và shared variables

webData đã dùng Mutex, nhưng nhiều biến actuator/metrics hiện là volatile và được đọc bởi Web/Display. Với project học tập điều này đơn giản, nhưng hệ thống lớn hơn nên cân nhắc snapshot/mutex/queue thống nhất.

14.4. Timing bằng tick có độ phân giải giới hạn

xTaskGetTickCount() phù hợp cho phép đo cấp tick. Nếu cần đo microsecond chính xác hơn có thể dùng timer có độ phân giải cao hơn, nhưng phải thiết kế đo không làm ảnh hưởng hệ thống.

14.5. Tách module là lợi ích gì?

Lỗi / thay đổi	Nơi cần xem trước
DHT/LDR đọc sai	sensors/
FSM chọn sai mode	decision/
Fan/relay/buzzer sai	control/
PIR phản hồi sai	pir/
OLED lỗi	display/
Web không nhận/gửi data	web/
Đổi pin / threshold / period	core/config.h

15. CÂU HỎI TỰ KIỂM TRA VÀ CÂU TRẢ LỜI BẢO VỆ NGẮN

Q: Tại sao dùng FreeRTOS thay vì một loop duy nhất?

A: Để tách sensing, decision, control, interrupt, display và web thành các task độc lập; scheduler quản lý priority và trạng thái BLOCKED/READY, giúp cấu trúc timing rõ hơn.

Q: Queue dùng để làm gì?

A: Truyền SensorData và DecisionData giữa các task theo cách thread-safe. Queue length 1 + overwrite giữ giá trị mới nhất.

Q: Semaphore dùng để làm gì?

A: Đồng bộ sự kiện PIR: ISR phát binary semaphore, PIRTask chờ semaphore và làm xử lý chính.

Q: Mutex khác Semaphore thế nào?

A: Mutex bảo vệ shared resource và mang ý nghĩa ownership; trong project nó bảo vệ webData. Semaphore PIR mang ý nghĩa event notification.

Q: Tại sao ISR phải ngắn?

A: ISR có thể làm trễ interrupt/task khác. Vì vậy ISR chỉ timestamp + GiveFromISR, còn xử lý chính giao cho PIRTask.

Q: DecisionTask có bật fan không?

A: Không. DecisionTask chỉ tạo SystemMode. ControlTask mới ánh xạ mode sang GPIO/PWM/Buzzer.

Q: Tại sao ControlTask priority cao hơn DisplayTask?

A: Actuator liên quan phản hồi điều khiển, còn OLED có thể trễ nhẹ mà không làm thay đổi logic hệ thống.

Q: Web Dashboard điều khiển fan trực tiếp không?

A: Không. Web chỉ cung cấp SensorData ở WEB mode; dữ liệu vẫn đi qua FSM và ControlTask.

Q: Humidity hiện dùng để làm gì?

A: Thu thập và hiển thị; chưa tham gia evaluateMode trong source hiện tại.

Q: 0 ms PIR latency có nghĩa là latency bằng 0 tuyệt đối?

A: Không. Có thể chỉ nhỏ hơn độ phân giải FreeRTOS tick được dùng để đo.

15.1. Câu mô tả project trong 30 giây

Câu trả lời mẫu: Hệ thống sử dụng ESP32-S3 làm bộ điều khiển trung tâm và FreeRTOS để chia chức năng thành các task độc lập. SensorTask thu thập dữ liệu và gửi qua sensorQueue. DecisionTask áp dụng FSM theo temperature, light và occupancy để xác định mode. ControlTask nhận mode và điều khiển relay, fan PWM và buzzer. PIR được xử lý bằng hardware interrupt; ISR chỉ phát binary semaphore để đánh thức PIRTask. DisplayTask và WebTask đảm nhiệm giao diện ở priority thấp hơn, đồng thời hệ thống ghi nhận một số timing metric để đánh giá khả năng phản hồi.

15.2. Thứ tự học lại source trước ngày bảo vệ

5. config.h -> nhớ GPIO, threshold, period.
6. types.h -> hiểu SensorData, SystemMode.
7. main.cpp -> hiểu setup và xTaskCreate.
8. sensors.cpp -> hiểu input và sensorQueue.

9. decision.cpp -> thuộc FSM priority.
10. control.cpp -> thuộc mapping mode -> actuator.
11. pir.cpp -> giải thích ISR + Semaphore + PIRTask.
12. app_state.cpp -> phân biệt Queue/Semaphore/Mutex.
13. web_server.cpp -> hiểu REAL/WEB pipeline.
14. real-time metrics -> hiểu đo cái gì và giới hạn của phép đo.

Giải thích bản chất: Nếu tự vẽ được hai sơ đồ: SensorTask -> Queue -> DecisionTask -> Queue -> ControlTask và PIR -> ISR -> Semaphore -> PIRTask, bạn đã nắm phần cốt lõi nhất của project.